

Python Programming Course

Table of Contents

Lecture 1: Introduction to Python Programming

Lecture 2: Data Types and Variables

Lecture 3: Control Structures: Loops and Conditionals

Lecture 1: Introduction to Python Programming

Welcome to the Introduction to Python Programming lecture. In this session, we will cover the fundamental concepts of Python programming, including source code, syntax, and the Python interpreter. By the end of this lecture, you will be able to write simple expressions and understand the output generated by the Python console.

What is Source Code?

Source code is the sequence of instructions that make up a program. It is written in a programming language, in this case, Python. The source code is what you, as a programmer, will write and edit to create your applications.

Understanding Syntax

Syntax refers to the set of legal structures and commands that can be used in a programming language. Each programming language has its own syntax rules that dictate how code must be written. In Python, for example, indentation is crucial as it defines the blocks of code.

The Python Interpreter

Python is an interpreted language, which means that the Python interpreter directly executes the source code without the need for a separate compilation step. This allows for an interactive environment where you can write and test code snippets quickly. The Python interpreter provides a console where you can enter commands and see immediate results.

Example of Using the Python Console

When you open the Python console, you will see a prompt (`>>>`) where you can enter expressions. For instance: - Entering `3 + 7` will output `10`. - Entering `3 < 15` will output `True`. - If you enter `'print me'`, it will return `'print me'`, while `print 'print me'` will display `print me` on the console.

Writing Simple Expressions

An expression is a combination of values and operations that computes a value. In Python, you can use various arithmetic operators: - `+` for addition - `-` for subtraction - `*` for multiplication - `/` for division - `%` for modulus (remainder) - `**` for exponentiation

Operator Precedence

Python follows a specific order of operations, known as precedence. For example, multiplication and division have higher precedence than addition and subtraction. Thus, in the expression `1 + 3 * 4`, the multiplication is performed first, resulting in `13`. You can use parentheses to enforce a specific order, such as in `(1 + 3) * 4`, which results in `16`.

Working with Numbers

Python can handle both integers and real numbers (floating-point numbers). For example: - Integer division: `35 / 5` results in `7`. - Real number division: `15.0 / 2.0` results in `7.5`. - The modulus operator can be used to find remainders, such as `14 % 4`, which results in `2`.

Using Variables

A variable is a named piece of memory that can store a value. You can assign a value to a variable using the assignment statement: - Example: `x = 5` or `gpa = 3.14`. Once a variable has been assigned a value, you can use it in expressions. For instance, if `x = 7`, then `x + 4` will yield `11`.

Outputting to the Console

To display output in Python, you can use the `print` function. The syntax is: - `print("Message")` or `print(Expression)`. You can also print multiple items on the same line by separating them with commas: - Example: `print("Hello, world!")` will output `Hello, world!`.

Getting User Input

You can read input from the user using the `input` function. For example: `- age = input("How old are you? ")` will prompt the user for their age and store it in the variable `age`.

Conclusion

In this lecture, we have introduced the basics of Python programming, including source code, syntax, and the Python interpreter. You have learned how to write simple expressions, use variables, and output results to the console. In the next lecture, we will delve deeper into control structures and data types in Python.

Quiz Questions

Q1: What is source code?

- The output of a program
- The sequence of instructions that make up a program
- The Python interpreter
- A type of variable

Correct: The sequence of instructions that make up a program

Q2: What does the Python interpreter do?

- Compiles source code into machine code
- Executes source code directly
- Translates Python code into Java
- Stores variables in memory

Correct: Executes source code directly

Q3: Which operator has higher precedence in Python?

- +
- -
- *
- %

*Correct: **

Lecture 2: Data Types and Variables

In this lecture, we will explore the fundamental concepts of data types and variables in Python. Understanding these concepts is crucial for effective programming and data manipulation. By the end of this lecture, you will be able to identify different data types, create and use variables, perform arithmetic operations, and utilize built-in functions for data manipulation.

1. Introduction to Data Types

Data types are classifications that specify which type of value a variable can hold. In Python, the primary data types include: - **Integers**: Whole numbers, e.g., 5, -3, 42. - **Floating-point numbers**: Numbers with decimal points, e.g., 3.14, -0.001, 2.0. - **Strings**: Sequences of characters enclosed in quotes, e.g., 'hello', "Python Programming".

2. Working with Variables

A variable is a named piece of memory that can store a value. To create a variable, you use an assignment statement, which has the syntax:

```
name = value
```

For example: - `x = 5` - `gpa = 3.14`

Once a variable is assigned a value, you can use it in expressions. For instance: - If `x = 7`, then `x + 4` evaluates to 11.

3. Performing Arithmetic Operations

Python supports several arithmetic operations, which can be performed on integers and floating-point numbers. The basic arithmetic operators include: - **Addition (+)**: Adds two numbers. - **Subtraction (-)**: Subtracts one number from another. - **Multiplication (*)**: Multiplies two numbers. - **Division (/)**: Divides one number by another, resulting in a floating-point number. - **Integer Division (//)**: Divides and returns the integer part of the quotient. - **Modulus (%)**: Returns the remainder of a division. - **Exponentiation (**)**: Raises a number to the power of another.

Example of Arithmetic Operations:

```
>>> 3 + 7 # Addition
10
>>> 5 - 2 # Subtraction
3
>>> 4 * 2 # Multiplication
8
>>> 8 / 2 # Division
4.0
>>> 7 // 2 # Integer Division
3
>>> 7 % 2 # Modulus
1
>>> 2 ** 3 # Exponentiation
8
```

4. Understanding Data Type Conversion

Python allows for conversion between data types. You can convert an integer to a float and vice versa using the `int()` and `float()` functions: - `int(2.5)` results in 2. - `float(2)` results in 2.0.

5. Strings in Python

Strings are a vital data type in Python. They can be created using single or double quotes. You can

also use triple quotes for multi-line strings. Here are some important string operations: - **Concatenation**: Combining two strings using the + operator. - **Indexing**: Accessing individual characters using square brackets, e.g., s[0] gives the first character. - **Slicing**: Extracting a substring using the syntax s[start:end].

Example of String Operations:

```
>>> s = 'hello'
>>> s + ' world' # Concatenation
'hello world'
>>> s[1] # Indexing
'e'
>>> s[1:4] # Slicing
'ell'
```

6. Built-in Functions for Data Manipulation

Python provides several built-in functions that are useful for data manipulation: - len(): Returns the length of a string or list. - str(): Converts a value to a string. - int(): Converts a value to an integer. - float(): Converts a value to a floating-point number.

Example of Built-in Functions:

```
>>> len(s) # Length of the string
5
>>> str(10) # Convert integer to string
'10'
>>> int('5') # Convert string to integer
5
>>> float('3.14') # Convert string to float
3.14
```

7. Conclusion

In this lecture, we covered the essential data types in Python, how to create and use variables, perform arithmetic operations, and utilize built-in functions for data manipulation. Understanding these concepts is foundational for further programming in Python. As you practice, try to create your own variables, perform operations, and manipulate strings to solidify your understanding.

Quiz Questions

Q1: What are the primary data types in Python?

- Integers, Strings, Lists
- Integers, Floating-point numbers, Strings
- Strings, Tuples, Dictionaries
- Booleans, Sets, Arrays

Correct: Integers, Floating-point numbers, Strings

Q2: Which operator is used for integer division in Python?

- /
- //
- %
- **

Correct: //

Q3: What function would you use to find the length of a string in Python?

- size()
- length()
- len()
- count()

Correct: len()

Lecture 3: Control Structures: Loops and Conditionals

In this lecture, we will explore control structures in Python, focusing on loops and conditional statements. Control structures are essential for creating dynamic and responsive code, allowing us to implement repetition and selection in our programs. By the end of this lecture, you will understand how to use 'for' and 'while' loops, as well as 'if', 'elif', and 'else' statements.

Introduction to Control Structures

Control structures are constructs that dictate the flow of control in a program. They allow us to execute certain blocks of code based on specific conditions or to repeat blocks of code multiple times. In Python, the primary control structures we will cover are loops and conditionals.

Loops

Loops are used to execute a block of code repeatedly. Python provides two types of loops: 'for' loops and 'while' loops.

For Loop

The 'for' loop is used to iterate over a sequence (like a list, tuple, or string) or a range of numbers. The syntax for a 'for' loop is as follows:

```
for variableName in groupOfValues:  
    statements
```

Example:

```
for x in range(1, 6):  
    print(x, "squared is", x * x)
```

Output:

```
1 squared is 1  
2 squared is 4  
3 squared is 9  
4 squared is 16  
5 squared is 25
```

The `range()` function generates a sequence of numbers. In the example above, it generates numbers from 1 to 5.

While Loop

The 'while' loop continues to execute a block of code as long as a specified condition is true. The syntax is:

```
while condition:  
    statements
```

Example:

```
number = 1  
while number < 200:  
    print(number)  
    number = number * 2
```

Output:

```
1  
2  
4
```

8
16
32
64
128

Conditionals

Conditional statements allow us to execute certain blocks of code based on whether a condition is true or false. The primary conditional statements in Python are 'if', 'elif', and 'else'.

If Statement

The 'if' statement executes a block of code if a specified condition is true. The syntax is:

```
if condition:  
    statements
```

Example:

```
gpa = 3.4  
if gpa > 2.0:  
    print("Your application is accepted.")
```

If/Else Statement

The 'if/else' statement allows us to execute one block of code if the condition is true and another block if it is false. The syntax is:

```
if condition:  
    statements  
else:  
    statements
```

Example:

```
gpa = 1.4  
if gpa > 2.0:  
    print("Welcome to Mars University!")  
else:  
    print("Your application is denied.")
```

Elif Statement

The 'elif' statement allows us to check multiple conditions. The syntax is:

```
if condition:  
    statements  
elif condition:  
    statements  
else:  
    statements
```

Example:

```
x = 30  
if x <= 15:  
    y = x + 15  
elif x <= 30:  
    y = x + 30  
else:  
    y = x
```



```
print('y =', y)
```

Combining Loops and Conditionals

Loops and conditionals can be combined to create more complex logic in your programs. For example, you can use a loop to iterate through a list of numbers and use a conditional statement to check if each number is even or odd.

Example:

```
for i in range(1, 11):  
    if i % 2 == 0:  
        print(i, "is even")  
    else:  
        print(i, "is odd")
```

Conclusion

In this lecture, we covered the fundamental control structures in Python: loops and conditionals. We learned how to use 'for' and 'while' loops to implement repetition and how to use 'if', 'elif', and 'else' statements to implement selection. Mastering these concepts is crucial for writing dynamic and responsive Python programs. In the next lecture, we will explore functions and how they can help us organize our code more effectively.

Quiz Questions

Q1: What are the two types of loops provided by Python?

- For and Until loops
- For and While loops
- Do While and For loops
- If and Else loops

Correct: For and While loops

Q2: Which statement is used to execute a block of code if a specified condition is true?

- elif
- else
- if
- while

Correct: if

Q3: What does the 'range()' function do in a for loop?

- Generates a random number
- Creates a list of strings
- Generates a sequence of numbers
- Counts the number of iterations

Correct: Generates a sequence of numbers